Стандарты кодирования

Стандарты кодирования (или **кодстайл**) — это набор правил и соглашений, которые помогают поддерживать код в читаемом и поддерживаемом состоянии. Стандарты кодирования особенно важны в команде, где несколько разработчиков работают над одним проектом. Они обеспечивают единообразие, что снижает вероятность ошибок и облегчает командную работу.

---

1. Кодстайл и линтинг

Кодстайл охватывает различные аспекты написания кода, такие как:

- Форматирование (отступы, пробелы).
- Именование переменных, функций и классов.
- Организация кода (разделение на модули, функции, классы и т.д.).
- Согласованность в использовании различных конструкций языка.

Линтинг — это процесс автоматической проверки исходного кода на соответствие установленным стандартам. Линтеры помогают находить ошибки, предупреждения и несоответствия стандартам кодирования.

Основные инструменты для линтинга:

1. **ESLint** — линтер для JavaScript, который помогает соблюсти правила кодирования, избегать ошибок и улучшать качество кода.
2. **Prettier** — форматтер кода, который автоматически форматирует код в соответствии с предустановленными правилами.
3. **Stylelint** — линтер для CSS и SCSS, который помогает поддерживать единообразие в стилях.

1.1 ESLint

ESLint — это статический анализатор для JavaScript, который проверяет код на ошибки и предупреждает о потенциальных проблемах. Он помогает стандартизировать код и находит такие ошибки, как неиспользуемые переменные, ошибки в синтаксисе, несоответствие стилей кодирования и т.д.

Как использовать ESLint:

1. Установите ESLint в проект:



Инициализируйте конфигурацию ESLint:



Настройте файл конфигурации `.eslintrc.js` или `.eslintrc.json` для задания правил, например:



Запуск линтинга



1.2 Prettier

Prettier — это инструмент для автоматического форматирования кода. В отличие от линтеров, которые сообщают о проблемах, Prettier **сам автоматически форматирует** код в соответствии с установленными правилами. Использование Prettier помогает поддерживать единообразие форматирования, особенно при работе с командами.

Как использовать Prettier:

Установите Prettier в проект:



Создайте файл конфигурации `.prettierrc` с вашими настройками



Запустите Prettier



Совмещение ESLint и Prettier: Для интеграции ESLint и Prettier в одном проекте можно использовать **eslint-plugin-prettier** и **eslint-config-prettier**. Эти плагины позволяют ESLint работать с Prettier и устранять конфликты между линтерами и форматтерами.

Пример конфигурации для ESLint и Prettier



В файле `.eslintrc.js` добавьте:



1.3 Stylelint

Stylelint — линтер для CSS, SCSS и других препроцессоров. Он помогает следить за кодом стилей, предотвращая ошибки и нарушения стайлгайдов.

Как использовать Stylelint:

1. Установите Stylelint:



Настройте конфигурацию в файле `.stylelintrc.json`



Запустите линтинг стилей



---

2. Советы по улучшению читаемости кода

Читаемость кода — это один из важнейших факторов, который определяет, насколько легко другие разработчики смогут понять, работать с вашим кодом и модифицировать его. Вот несколько рекомендаций по улучшению читаемости кода:

2.1 Именование переменных и функций

Именование — это основа хорошей читаемости. Имена переменных и функций должны быть **ясными и описательными**.

- **Переменные**: Имена переменных должны отражать их значение. Например, вместо `x` используйте `userAge`, а вместо `temp` — `temperature`.
- **Функции**: Функции должны начинаться с глаголов, чтобы ясно отражать их действия, например: `getUserData()`, `calculateTotal()`, `saveData()`.
- **Ясные имена классов CSS**: Применяйте описательные и семантические названия классов. Например, вместо `.btn1` лучше использовать `.btn-primary` или `.btn-`

`secondary`.

- **Кодирование в едином стиле**: Придерживайтесь единого стиля именования (CamelCase, kebab-case и т.д.) на протяжении всего проекта.

Пример плохого именования:



Пример хорошего именования:



2.2 Использование стрелочных функций или обычных функций

Выбор между **стрелочными функциями** и **обычными функциями** зависит от контекста, но стрелочные функции часто **короче** и **удобнее** для работы с лексическим контекстом `this`.

- **Стрелочные функции**:
  - Простой синтаксис.
  - Лексическое связывание `this`.

Пример:



- **Обычные функции**:
  - Лучше подходят для методов объектов и конструкций, где требуется явное связывание `this`.

Пример:



2.3 Использование строгих типов и дескрипторов свойств

Использование **строгой типизации** (например, через TypeScript) помогает избежать ошибок, связанных с несовпадением типов, и делает код более предсказуемым и читаемым.

- **TypeScript** позволяет определять типы для переменных и аргументов функций, что предотвращает ошибки на этапе компиляции.
- **Descriptors** (например, `Object.defineProperty`) дают возможность управлять поведением свойств объектов.

Пример строгой типизации (TypeScript):



2.4 Документирование кода

Хорошо документированный код значительно повышает его читаемость.

- **Комментирование кода**: Добавляйте комментарии к сложным участкам кода, чтобы объяснить логику или назначение. Это поможет вам и другим разработчикам в будущем.
- **Документация стилей**: Используйте README файлы или другие формы документации, чтобы описать структуру стилей и использование классов.

Используйте **JSDoc** для документирования функций и классов.

Пример документации с JSDoc:



2.5 Разделение на небольшие функции

Не пишите слишком большие функции. Чем меньше функция, тем легче её понять и поддерживать. Разделяйте логику на маленькие, однотипные функции, каждая из которых решает одну задачу.

Пример:



---

3. Использование правил и линтеров в CI/CD

Для того чтобы обеспечить **единообразие** кодирования в команде, настройте линтеры и форматтеры в вашем CI/CD процессе. Это позволяет автоматически проверять код на стандарты и стиль до его слияния с основной веткой.

Пример настройки линтеров в CI/CD с GitHub Actions:

